Human Verification: `Containers.Basic`

Containers, Extensions, and Morphisms

Machine-generated from `lean/Basic.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Basic` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 Container and its extension **2**
2.1 Structure: `Container` . . . 2
2.2 Definition: `Ext` . . . 2
2.3 Definition: `Ext.map` . . . 2
2.4 Theorem: `Ext.map_id` . . . 3
2.5 Theorem: `Ext.map_comp` . . . 3

3 Container morphisms **3**
3.1 Structure: `ContainerMorphism` . . . 3
3.2 Definition: `ContainerMorphism.toNat` . . . 4
3.3 Theorem: `ContainerMorphism.toNat_natural` . . . 4

4 The representation theorem **5**
4.1 Structure: `ExtNatTrans` . . . 5
4.2 Definition: `ContainerMorphism.toExtNatTrans` . . . 5
4.3 Definition: `ExtNatTrans.toMorphism` . . . 5
4.4 Theorem: `toExtNatTrans_toMorphism` (morphism round-trip) . . . 6
4.5 Theorem: `toMorphism_toExtNatTrans` (natural transformation round-trip) . . . 6

5 Translation conventions **7**

1 Overview

This file develops the basic theory of *containers* (Abbott, Altenkirch, Ghani) using only Lean 4 core — no Mathlib dependency.

A container is a type of *shapes* together with, for each shape, a type of *positions*. Its *extension* is the polynomial (dependent-pair) functor it induces on **Type**. Container morphisms induce

natural transformations between extensions, and — this is the representation theorem — every natural transformation arises this way, uniquely.

Everything is **Type**-level (a single universe) to keep the development self-contained.

2 Container and its extension

2.1 Structure: `Container`

Definition 2.1 (Container)**.** A **container** $C = (S, P)$ consists of a type S of *shapes* and, for each shape $s : S$, a type $P(s)$ of *positions*.

Lean.
```
structure Container where
  Shape : Type
  Pos : Shape → Type
```

Haskell.
```
-- In Lean, Pos is a dependent type family  Pos : Shape → Type.
-- In Haskell we approximate this by fixing both type parameters.
-- A "container" is then the *pair* of types (s, p), where s is the
-- type of shapes and p is the type of positions.
```

Fidelity note. In the Lean formalisation, the position type $P(s)$ varies with the shape s; this is a genuine dependent type. In our Haskell rendering, positions are a single type parameter `p`, which is the standard approximation for non-dependently-typed languages. All subsequent definitions inherit this simplification.

2.2 Definition: `Ext`

Definition 2.2 (Extension)**.** The **extension** of a container $C = (S, P)$ at a type X is the dependent sum

$$\mathrm{Ext}\, C\, X \;=\; \Sigma_{s:S}\,(P(s) \to X).$$

An element is a shape s together with a labelling of its positions by elements of X. This is the polynomial functor associated to C.

Lean.
```
def Ext (C : Container) (X : Type) : Type :=
  (s : C.Shape) * (C.Pos s → X)
```

Haskell.
```
data Ext s p x = Ext s (p → x)
```

2.3 Definition: `Ext.map`

Definition 2.3 (Functorial action)**.** Given $f : X \to Y$, the **map** $\mathrm{Ext}\, C\, f : \mathrm{Ext}\, C\, X \to \mathrm{Ext}\, C\, Y$ relabels positions by post-composing the labelling function with f:

$$\mathrm{Ext}\, C\, f\, \langle s, v\rangle \;=\; \langle s,\; f \circ v\rangle.$$

Lean.
```
def Ext.map {C : Container} {X Y : Type} (f : X → Y) :
    Ext C X → Ext C Y :=
  fun p ⇒ ⟨p.1, fun i ⇒ f (p.2 i)⟩
```

Haskell.

```haskell
instance Functor (Ext s p) where
  fmap f (Ext s v) = Ext s (f . v)
```

2.4 Theorem: `Ext.map_id`

Theorem 2.4 (Identity law)**.** For every container C and type X,

$$\operatorname{Ext} C \, \mathrm{id}_X \;=\; \mathrm{id}_{\operatorname{Ext} C \, X}.$$

That is, the functorial action of $\operatorname{Ext} C$ preserves the identity function.

Lean.

```
theorem Ext.map_id {C : Container} {X : Type} :
    Ext.map (C := C) (id : X → X) = id
```

Haskell.

```haskell
-- Functor law (identity):
-- fmap id = id
--
-- For all (Ext s v):  fmap id (Ext s v) = Ext s (id . v) = Ext s v
```

2.5 Theorem: `Ext.map_comp`

Theorem 2.5 (Composition law)**.** For every container C and functions $f : X \to Y$, $g : Y \to Z$,

$$\operatorname{Ext} C \, (g \circ f) \;=\; \operatorname{Ext} C \, g \;\circ\; \operatorname{Ext} C \, f.$$

That is, the functorial action of $\operatorname{Ext} C$ preserves composition.

Lean.

```
theorem Ext.map_comp {C : Container} {X Y Z : Type}
    (f : X → Y) (g : Y → Z) :
    Ext.map (C := C) (g  ∘  f) = Ext.map g  ∘  Ext.map f
```

Haskell.

```haskell
-- Functor law (composition):
-- fmap (g . f) = fmap g . fmap f
--
-- For all (Ext s v):
--   fmap (g . f) (Ext s v) = Ext s (g . f . v)
--                          = fmap g (Ext s (f . v))
--                          = (fmap g . fmap f) (Ext s v)
```

3 Container morphisms

3.1 Structure: `ContainerMorphism`

Definition 3.1 (Container morphism)**.** A **morphism of containers** $\varphi : C \Rightarrow D$ between $C = (S_C, P_C)$ and $D = (S_D, P_D)$ consists of:

- a map on shapes $u : S_C \to S_D$, and
- a *contravariant* map on positions: for each $s : S_C$, a function $g_s : P_D(u(s)) \to P_C(s)$.

The position map goes "backwards": it pulls D-positions over the image shape back to C-positions.

Lean.

```
structure ContainerMorphism (C D : Container) where
  onShapes : C.Shape → D.Shape
  onPos : (s : C.Shape) → D.Pos (onShapes s) → C.Pos s
```

Haskell.

```
data Morph s p t q = Morph
  { onShapes :: s → t
  , onPos    :: s → q → p
  }
```

Fidelity note. In Lean, `onPos` has a dependent type: given $s : S_C$, it maps positions of the *image* shape $u(s)$ back to positions of s. In Haskell, the type `s -> q -> p` does not capture this dependency; it simply maps an `s` and a `q` to a `p`. The intent is the same, but the type-level guarantee that `q` refers to positions of the image of `s` is lost.

3.2 Definition: `ContainerMorphism.toNat`

Definition 3.2 (Induced natural transformation)**.** A container morphism $\varphi = (u, g)$ induces, at every type X, a map

$$\varphi^*_X : \mathrm{Ext}\, C\, X \;\to\; \mathrm{Ext}\, D\, X, \qquad \varphi^*_X\langle s, v\rangle \;=\; \langle u(s),\; v \circ g_s\rangle.$$

The family $(\varphi^*_X)_X$ is a natural transformation $\mathrm{Ext}\, C \Longrightarrow \mathrm{Ext}\, D$.

Lean.

```
def ContainerMorphism.toNat {C D : Container}
    (phi : ContainerMorphism C D)
    (X : Type) : Ext C X → Ext D X :=
  fun p ⇒ ⟨phi.onShapes p.1, fun i ⇒ p.2 (phi.onPos p.1 i)⟩
```

Haskell.

```
toNat :: Morph s p t q → Ext s p x → Ext t q x
toNat (Morph u g) (Ext s v) = Ext (u s) (v . g s)
```

3.3 Theorem: `ContainerMorphism.toNat_natural`

Theorem 3.3 (Naturality)**.** For every container morphism $\varphi : C \Rightarrow D$ and every function $f : X \to Y$, the following square commutes:

$$\mathrm{Ext}\, D\, f \;\circ\; \varphi^*_X \;=\; \varphi^*_Y \;\circ\; \mathrm{Ext}\, C\, f.$$

Equivalently, the family $(\varphi^*_X)_X$ is a natural transformation.

Lean.

```
theorem ContainerMorphism.toNat_natural {C D : Container}
    (phi : ContainerMorphism C D) {X Y : Type} (f : X → Y) :
    Ext.map f  ∘  phi.toNat X = phi.toNat Y  ∘  Ext.map f
```

Haskell.

```
-- Naturality:
-- fmap f . toNat phi = toNat phi . fmap f
--
-- For all (Ext s v):
--   (fmap f . toNat (Morph u g)) (Ext s v)
--     = fmap f (Ext (u s) (v . g s))
--     = Ext (u s) (f . v . g s)
--     = toNat (Morph u g) (Ext s (f . v))
--     = (toNat (Morph u g) . fmap f) (Ext s v)
```

4 The representation theorem

Container morphisms $C \Rightarrow D$ are in bijection with natural transformations $\mathrm{Ext}\, C \Longrightarrow \mathrm{Ext}\, D$. The key idea: a natural transformation is determined by where it sends the *generic element* $\langle s, \mathrm{id}\rangle \in \mathrm{Ext}\, C\, (P_C(s))$. Evaluating at $X := P_C(s)$ recovers the shape map; naturality (a Yoneda-style argument) recovers the position map.

4.1 Structure: `ExtNatTrans`

Definition 4.1 (Natural transformation between extensions)**.** A **natural transformation** $\eta : \mathrm{Ext}\, C \Longrightarrow \mathrm{Ext}\, D$ is a family of maps $\eta_X : \mathrm{Ext}\, C\, X \to \mathrm{Ext}\, D\, X$, indexed by types X, satisfying the naturality condition: for every $f : X \to Y$,

$$\mathrm{Ext}\, D\, f \circ \eta_X = \eta_Y \circ \mathrm{Ext}\, C\, f.$$

Lean.

```
structure ExtNatTrans (C D : Container) where
  component : (X : Type) → Ext C X → Ext D X
  natural : ∀ {X Y : Type} (f : X → Y),
    Ext.map f ∘ component X = component Y ∘ Ext.map f
```

Haskell.

```
-- A natural transformation between container extensions.
-- In Haskell, the naturality condition is a free theorem
-- (parametricity), so we only need the polymorphic function.
type ExtNatTrans s p t q = forall x. Ext s p x → Ext t q x
```

Fidelity note. In Haskell, a polymorphic function `forall x. Ext s p x -> Ext t q x` is automatically natural by parametricity (the "free theorem"). In Lean, naturality must be stated and proved explicitly, since Lean's type theory does not give you parametricity for free. The Lean structure bundles the proof; the Haskell type relies on it implicitly.

4.2 Definition: `ContainerMorphism.toExtNatTrans`

Definition 4.2 (Forward direction)**.** Every container morphism $\varphi : C \Rightarrow D$ gives rise to a natural transformation $\varphi^* : \mathrm{Ext}\, C \Longrightarrow \mathrm{Ext}\, D$, by bundling `toNat` with its naturality proof.

Lean.

```
def ContainerMorphism.toExtNatTrans {C D : Container}
    (phi : ContainerMorphism C D) : ExtNatTrans C D where
  component := phi.toNat
  natural := fun f ⇒ phi.toNat_natural f
```

Haskell.

```
toExtNatTrans :: Morph s p t q → (forall x. Ext s p x → Ext t q x)
toExtNatTrans = toNat
-- Naturality is automatic by parametricity.
```

4.3 Definition: `ExtNatTrans.toMorphism`

Definition 4.3 (Backward direction)**.** Given a natural transformation $\eta : \mathrm{Ext}\, C \Longrightarrow \mathrm{Ext}\, D$, recover a container morphism by probing η at the *generic element* $\langle s, \mathrm{id}\rangle \in \mathrm{Ext}\, C\, (P_C(s))$:

$$\begin{aligned} u(s) &= \pi_1(\eta_{P_C(s)}\langle s, \mathrm{id}\rangle), \\ g_s &= \pi_2(\eta_{P_C(s)}\langle s, \mathrm{id}\rangle). \end{aligned}$$

The shape map is the first component of the result; the position map is the second.

Lean.

```
def ExtNatTrans.toMorphism {C D : Container}
    (eta : ExtNatTrans C D) : ContainerMorphism C D where
  onShapes := fun s ⇒ (eta.component (C.Pos s) ⟨s, id⟩).1
  onPos := fun s ⇒ (eta.component (C.Pos s) ⟨s, id⟩).2
```

Haskell.

```
toMorphism :: (forall x. Ext s p x → Ext t q x) → Morph s p t q
toMorphism eta = Morph u g
  where
    -- Probe at the generic element (s, id) :: Ext s p p
    -- This only type-checks when s is a concrete shape value,
    -- so in practice we'd need to supply a witness.
    Ext t' g' = eta (Ext s id)   -- for each shape s
    u s = t'
    g s = g'
```

Fidelity note. The Lean version probes η at $X = P_C(s)$ — a type that *depends on* the shape s. In Haskell, the rank-2 type `forall x. Ext s p x -> Ext t q x` can be instantiated at $x = p$ (the position type), and the "generic element" is `Ext s id`. The dependent aspect — that we choose a *different* X for each s — is not expressible in Haskell's type system, but the Yoneda-style argument carries through operationally.

4.4 Theorem: `toExtNatTrans_toMorphism` (morphism round-trip)

Theorem 4.4 (Morphism round-trip)**.** Starting from a container morphism φ, building the natural transformation φ^*, and reading a morphism back, returns the original:

$$\text{toMorphism}(\text{toExtNatTrans}(\varphi)) \;=\; \varphi.$$

This direction is purely computational (no appeal to naturality needed).

Lean.

```
theorem ContainerMorphism.toExtNatTrans_toMorphism
    {C D : Container}
    (phi : ContainerMorphism C D) :
    phi.toExtNatTrans.toMorphism = phi
```

Haskell.

```
-- Round-trip (morphism direction):
-- toMorphism (toExtNatTrans phi) = phi
--
-- Proof sketch:
--   toExtNatTrans (Morph u g) = \(Ext s v) → Ext (u s) (v . g s)
--   Probing at (Ext s id) gives Ext (u s) (id . g s) = Ext (u s) (g s
    )
--   So toMorphism recovers (Morph u g).
```

4.5 Theorem: `toMorphism_toExtNatTrans` (natural transformation round-trip)

Theorem 4.5 (Natural transformation round-trip — the representation theorem)**.** Starting from a natural transformation η, extracting a morphism, and building a natural transformation back, returns the original:

$$\text{toExtNatTrans}(\text{toMorphism}(\eta)).\text{component} \;=\; \eta.\text{component}.$$

This is the substantive direction. It requires the naturality law: for $p = \langle s, g\rangle : \mathrm{Ext}\, C\, X$, we apply naturality of η at g to the generic element $\langle s, \mathrm{id}\rangle$ and use $\mathrm{Ext}\, C\, g\, \langle s, \mathrm{id}\rangle = \langle s, g\rangle$ to recover $\eta_X \langle s, g\rangle$ from $\mathrm{Ext}\, D\, g\, (\eta_{P_C(s)} \langle s, \mathrm{id}\rangle)$.

Together with the morphism round-trip, this establishes a bijection:

$$\text{ContainerMorphism}(C, D) \;\cong\; \text{NatTrans}(\text{Ext}\, C, \text{Ext}\, D).$$

Lean.

```
theorem ExtNatTrans.toMorphism_toExtNatTrans
    {C D : Container}
    (eta : ExtNatTrans C D) :
    eta.toMorphism.toExtNatTrans.component = eta.component
```

Haskell.

```
-- Round-trip (natural transformation direction):
-- toExtNatTrans (toMorphism eta) = eta
--
-- This is the representation theorem.  In Haskell, it follows from
-- parametricity: a polymorphic function  forall x. F x → G x  is
-- uniquely determined by its action on the "identity" labelling.
-- In Lean, where parametricity is not built in, this requires an
-- explicit naturality argument (a Yoneda-style calculation).
```

5 Translation conventions

The following conventions apply throughout this document series.

1. **Dependent position families.** In Lean, a container has Pos : Shape → **Type** — positions depend on the shape. In Haskell, we approximate this with a single type parameter `p`. This is the standard simplification; all theorem statements remain meaningful, but type-level guarantees about which positions belong to which shapes are lost.

2. **Transport / coercion.** Several Lean definitions and theorems involve *transport* (▶ in Lean) along equality proofs between shapes. These have no Haskell counterpart and are flagged individually in fidelity notes. They will appear prominently in the directed-container file (`Directed.lean`), where laws D2 and D5 transport positions along shape equalities D1 and D4.

3. **Theorems as properties.** Lean theorems become either Haskell property functions (executable QuickCheck-style propositions) or equational laws stated in comments when the statement quantifies over types in a way that Haskell cannot express.

4. **Naturality for free.** In Haskell, polymorphic functions are natural by parametricity. In Lean, naturality is an explicit proof obligation. When a Lean structure bundles a naturality proof, the Haskell rendering simply gives the polymorphic type.

5. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.